

Clase del M4

Sitio: [Campus - Sadosky Capacitaciones](#)
Curso: Introducción a la administración de clústeres HPC_V1
Libro: Clase del M4

Imprimido por: Pablo Carrai
Día: miércoles, 17 de junio de 2026, 16:22

Tabla de contenidos

- 4.1 La arquitectura de cohesión: el rol del middleware en HPC
- 4.2 Caracterización de cargas de trabajo y diseño de particiones
- 4.3 Software disponible para gestión de colas/particiones
 - Particiones y comandos
- 4.5 Gestión del entorno: módulos de entorno
- 4.5 Contenedores en HPC: la era de la portabilidad
- Cierre

4.1 La arquitectura de cohesión: el rol del middleware en HPC

Como ya señalamos anteriormente, un clúster de computadoras es esencialmente un grupo de máquinas interconectadas, a menudo homogéneas o compatibles, que están configuradas y gestionadas para trabajar de manera coordinada y cohesiva como si fueran una única y potente unidad de procesamiento. Esta arquitectura permite que la carga de trabajo se distribuya, lo que resulta en una capacidad de procesamiento, almacenamiento y tolerancia a fallos muy superior a la que podría ofrecer cualquiera de sus componentes individuales.

¿Cómo se logra la unificación de recursos a nivel técnico? Como hemos visto en los módulos anteriores, la clave reside en una compleja interacción de hardware de alta velocidad, sistemas operativos especializados (a menudo distribuciones de Linux optimizadas), *middleware* de gestión de recursos y, fundamentalmente, protocolos de comunicación y programación específicos.



En este módulo nos centraremos en el *middleware*

El proceso de envío de trabajos

Supongamos que somos un usuario que quiere realizar un cálculo entre muchos hilos utilizando MPI y tiene acceso a un clúster HPC. ¿Cómo lo resolveríamos?

Veamos los pasos en el siguiente recurso interactivo:



El concepto de *middleware*

Como vimos en el recurso, para el usuario la acción es un simple comando de envío. Sin embargo, en el clúster se inicia un proceso que unifica un conjunto de computadoras independientes para dedicarlas exclusivamente a la resolución de su problema. El principal responsable de que esto suceda es el *middleware* de gestión de recursos, que tiene dos grandes responsabilidades:

Gestión de recursos o *resource management*: asignar nodos a un trabajo.

Planificación del trabajo o *job scheduling*: decidir qué trabajo se ejecuta y cuándo basándose en criterios de prioridad.

Como ya vimos en el módulo 2, el *scheduler* (planificador) es el componente de software encargado de decidir cómo

distribuir los recursos finitos (CPU, RAM, GPU, licencias) que gestiona el *Resource Manager*, entre una cola de solicitudes potencialmente infinita. Matemáticamente, el *scheduler* es el que resuelve el problema de optimización para determinar cuándo y dónde debe ejecutarse cada trabajo, mientras que el *resource manager* es el que ejecuta y monitorea esa asignación en los nodos físicos



Los usuarios envían *jobs* (scripts) solicitando N CPU y X GB de RAM por un tiempo T. El *middleware* pone estos trabajos en cola y los despacha cuando los recursos están libres.

4.2 Caracterización de cargas de trabajo y diseño de particiones

Gestión de trabajos

La gestión de la carga de trabajo requiere comprender la naturaleza de las tareas que realizan los usuarios y el comportamiento de dichos procesos con el fin de optimizar la configuración del *scheduler*.

La clasificación de las tareas o trabajos permite el establecimiento de particiones o colas especializadas. Por ejemplo, podríamos tener:

Tipo de trabajo	Características	Necesidades de infraestructura
Serial	1 núcleo, 1 nodo.	Alta frecuencia de reloj (GHz).
Paralelo SMP/OpenMP	Muchos núcleos, 1 nodo.	Memoria compartida grande, muchos cores por socket.
Paralelo MPI	Muchos núcleos, múltiples nodos.	Interconexión de baja latencia (InfiniBand/Omni-Path).
High Throughput (HTC)	Miles de trabajos cortos y pequeños.	<i>Scheduling</i> rápido, <i>throughput</i> de I/O.
GPU / AI	Entrenamiento de modelos, simulación MD.	GPU potentes (A100/H100), NVLink.



En este contexto partición y cola son sinónimos. *Particiones* se llaman en la terminología de Slurm. En nuestro rol de *cluster admin* debemos familiarizarnos con ambos términos.

Definir las colas o particiones es quizás la decisión técnica y estratégica más importante que toma el administrador de un clúster. Una mala estrategia puede dejar recursos ociosos mientras los usuarios esperan; una buena estrategia maximiza la ocupación del hardware.

Aquí presentamos 3 enfoques principales para diseñar colas en un entorno HPC:

1. Estrategia basada en el hardware (física)

Esta es la más intuitiva. Se crean colas separadas según el tipo de equipamiento físico. Esto evita que un usuario envíe un trabajo que solo requiere una CPU básica a un nodo costoso con 4 GPU, desperdiciando ese recurso.

2. Estrategia basada en tiempo y dimensionamiento (políticas de uso)

Aquí es donde se optimiza el flujo de trabajo. El objetivo es evitar que trabajos pequeños queden atrapados detrás de simulaciones de tres semanas.



Esta separación ayuda al *scheduler* a hacer *backfilling*. Si hay un hueco de 1 hora en los CPU antes de que empiece un trabajo gigante, Slurm puede colar un trabajo de la cola debug en ese hueco, aumentando la eficiencia total.

3. Estrategia de superposición con desalojo (*overlapping* y *preemption*)

Esta es una técnica avanzada que optimiza el uso de recursos ociosos. Se basa en que un mismo nodo físico pertenezca a varias particiones simultáneamente, pero con reglas de *preemption* (desalojo).

Escenario

Un laboratorio compró 5 nodos (son los dueños y, por lo tanto, tienen prioridad de uso), pero aceptan que otras unidades de la universidad los utilice (uso público) solo cuando ellos no los necesitan.

El mecanismo funciona del siguiente modo: si un usuario público está corriendo un cálculo y llega el propietario, el *scheduler* desaloja automáticamente al trabajo público (lo cancela o lo devuelve a la cola en pausa/*requeue*) para cederle el *hardware* al propietario al instante.

Recomendaciones

El problema central en HPC es el desperdicio de recursos, frecuentemente causado por solicitudes de recursos que superan las necesidades reales de las cargas de trabajo.

En el siguiente recurso interactivo se presentan estrategias clave para gestionar y minimizar este desperdicio:

Estrategias para la optimización de
recursos en HPC

SADOSKY
CAPACITACIONES



4.3 Software disponible para gestión de colas/particiones

Existen diversos sistemas para la gestión de colas, cada uno de los cuales veremos en detalle a continuación.

Torque/PBS

Torque (una versión comunitaria del original PBS, *Portable Batch System*) es un sistema maduro y robusto que dominó la supercomputación durante décadas. Aunque hoy en día está siendo reemplazado masivamente por Slurm, es importante conocerlo porque gran parte de la terminología actual y muchos scripts de usuarios antiguos (#PBS) provienen de aquí. A diferencia de sistemas más modernos, que tienden a ser monolíticos, la arquitectura de Torque se basa en la separación clara de responsabilidades en tres componentes principales:

PBS Pro es la versión comercial y OpenPBS la versión de código abierto de PBS, compitiendo con Torque y Slurm.

Sun Grid Engine (SGE)

Es el "tercer grande" en la historia de los *schedulers*. Aunque hoy en día Slurm domina el Top500 y las instalaciones modernas, SGE (y sus variantes como Open Grid Engine o Univa/Altair Grid Engine) sigue siendo muy común en la industria de la bioinformática y en clústeres empresariales heredados.

Al igual que Torque, SGE tiene una arquitectura clásica distribuida, pero su terminología es ligeramente diferente.

SLURM (*Simple Linux Utility for Resource Management*)

Es el más usado en el Top500. Opera principalmente con un modelo *controller/worker* (controlador/trabajador), apoyado por un demonio (*daemon*) de base de datos.

Particiones y comandos

Particiones en SLURM

Las entidades que maneja SLURM son las particiones.



Una partición es una agrupación lógica de nodos que comparten una configuración o política específica. En otros gestores se llaman explícitamente *queues* o colas, pero Slurm utiliza el término *partition* porque un mismo nodo puede pertenecer a varias particiones a la vez.

Cuando un administrador crea una partición en el archivo `slurm.conf`, define las reglas para esa cola.

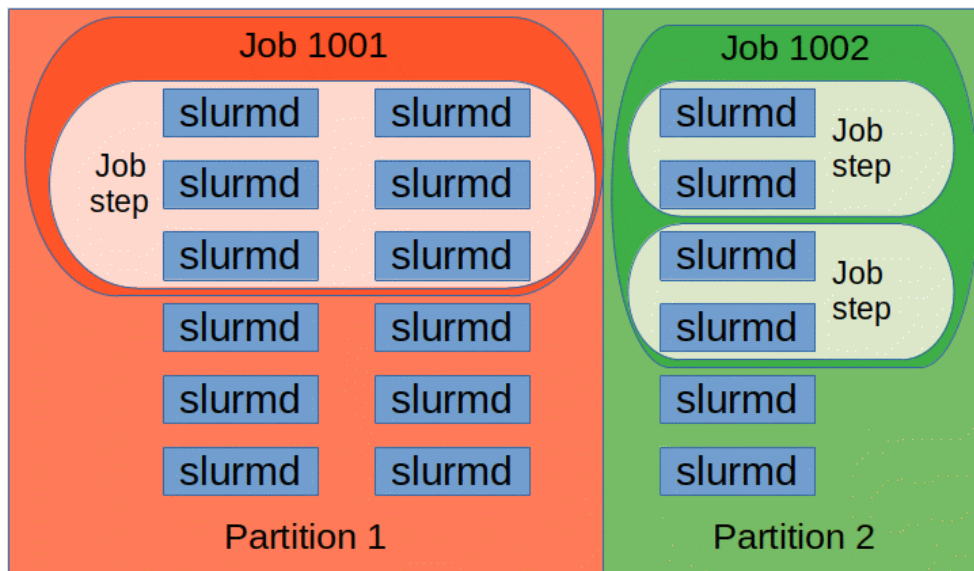


Diagrama de particiones en SLURM. De "Quick Start User Guide," por SchedMD, 2026, Slurm Workload Manager (<https://slurm.schedmd.com/quickstart.html>).

Comandos clave

Veamos algunos comandos clave:

```
# ver particiones disponibles
sinfo
```

Veamos un ejemplo de la información que nos brinda. La columna PARTITION muestra los nombres y si tienen un * indica que es la default:

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
debug*	up	1:00:00	4	idle	node[1-4]
gpu	up	3-00:00:00	8	mix	gpu[1-5],gpu[6-8]
bigmem	up	7-00:00:00	2	alloc	bigmem[1-2]
login	up	infinite	2	idle	login[1-2]

```
# Enviar un trabajo a una partición
sbatch --partition=gpu mi_script.sh
```

```
# o en su forma corta
sbatch -p gpu mi_script.sh
```

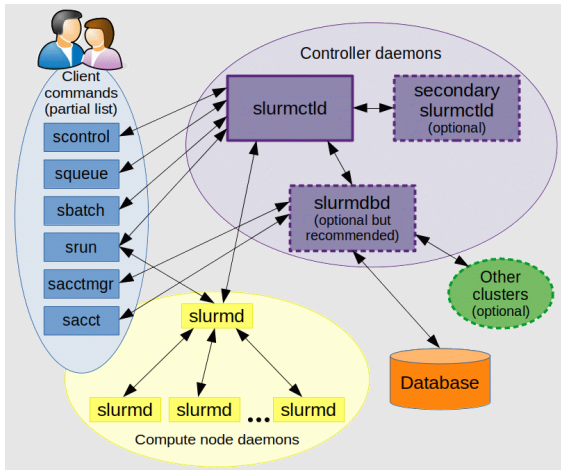
```
# Ver qué trabajos existen en el sistema
squeue
```

Vemos la información que nos brinda este comando:

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
65646	batch	chem	mike	R	24:19	2	node[7-8]
65647	batch	bio	joan	R	0:09	1	node14

65648	batch	math	phil	PD	0:00	6	(Resources)
-------	-------	------	------	----	------	---	-------------

La columna ST nos dice en qué estado está el job. PD significa que está pendiente de ejecución. Las razones típicas para los trabajos pendientes son recursos (esperando que los recursos estén disponibles) y prioridad (en cola detrás de un trabajo de mayor prioridad).



Componentes de SLURM. De "Quick Start User Guide," por SchedMD, 2026, Slurm Workload Manager (<https://slurm.schedmd.com/quickstart.html>)

En la figura de la izquierda podemos ver los componentes de SLURM.

El resto de los comandos de usuario incluyen:

sacct	sacctmgr	salloc	sattach
sbcast	scancel	scrontab	sdiag
sh5util	sprio	sreport	srun
sshare	sstat	strigger	sview

Todos los comandos pueden ejecutarse en cualquier parte del clúster.



Existen páginas man (*manual pages*) para todos los demonios, comandos y funciones API de Slurm. La opción de comando `--help` también proporciona un breve resumen de las opciones.

Diferencia conceptual crítica: *parallel environments* (PE)

En SLURM, si el usuario pide `--nodes=2 --ntasks=16`, el sistema asume que quiere correr algo paralelo y le da los recursos. Sin embargo, en SGE (*Sun Grid Engine*) esto no funciona así: si se envía un script pidiendo 16 cores sin especificar un PE, SGE a veces asignará 16 *cores* en máquinas aleatorias sin garantizar que estén conectadas o reservadas correctamente para MPI.



En SGE el concepto de paralelismo no es nativo por defecto, por lo que hay que definir explícitamente un entorno paralelo o *parallel environment* (PE).

Comandos de usuario

Existen diferentes comandos que usan los usuarios del clúster para interactuar con el sistema de colas. Cada sistema tiene su conjunto de comandos y directivas propio para subir un *job*, ver el estado de los trabajos enviados, ver nodos disponibles, etc.

Veamos en el siguiente cuadro una comparación entre comandos de usuario en Slurm, Torque/PBS y SGE:

Característica	Slurm	Torque/PBS	SGE
Script de envío	sbatch	qsub	qsub

Ver cola	<code>squeue</code>	<code>qstat</code>	<code>qstat</code>
Borrar trabajo	<code>scancel</code>	<code>qdel</code>	<code>qdel</code>
Directiva en script	<code>#SBATCH</code>	<code>#PBS</code>	<code>##</code>
Solicitar <i>shell</i> interactivo (ver nota de buenas prácticas)	<code>srunc --pty bash</code>	<code>qsub -l</code>	<code>qlogin</code> o <code>qrsh</code>
Ver info de nodos	<code>sinfo / scontrol</code>	<code>pbsnodes</code>	<code>qhost</code>
Archivo de salida	<code>slurm-%j.out</code>	<code><job_name>.o<id></code>	<code><job_name>.o<id></code>
Variables de entorno	<code>SLURM_JOB_ID</code>	<code>PBS_JOBID</code>	<code>JOB_ID</code>

¿Cuándo usar una *shell* interactiva? Muchos usuarios intentan compilar código o ejecutar pruebas rápidas directamente en el nodo de acceso (*login node*). Esto es una mala práctica porque ralentiza el sistema para todos los usuarios.

Buena práctica de administración:



Si el usuario necesita una terminal para probar comandos manualmente, debe solicitar una sesión interactiva: `srunc --pty /bin/bash`

Esto le dará una terminal, pero estará "dentro" de un nodo de cómputo, usando recursos asignados legalmente por el planificador.

4.5 Gestión del entorno: módulos de entorno

En un clúster compartido por cientos de usuarios surgen conflictos inevitables. Veamos uno de ellos y su solución.

El Problema de las dependencias

- El Grupo A necesita **GCC 9.0** para compilar un código antiguo de climatología.
- El Grupo B necesita **GCC 11.2** para usar las últimas funciones de C++20 en su IA.
- El SysAdmin no puede instalar ambos en `/usr/bin/gcc` porque el sistema operativo colapsaría o no sabría cuál ejecutar.

Para abordar este problema es necesario también tener en cuenta una regla importante:



En HPC, el software científico **NO** se instala en las rutas del sistema operativo. Se instala en rutas no estándar (ej: `/apps/software/gcc/9.0/` y `/apps/software/gcc/11.2/`)

Para este tipo de problemas, la solución es la modificación dinámica del entorno. Para ello necesitaremos una herramienta específica:

Environment Modules (Lmod) constituye una herramienta que posibilita la modificación dinámica del entorno de la *shell* del usuario.



- Al cargar el módulo "Python 3.9", el entorno de la terminal queda configurado para utilizar Python 3.9.
- Al descargar dicho módulo e insertar "Python 2.7", la terminal pasa a emplear Python 2.7.

Es importante destacar que este proceso se desarrolla sin generar afectaciones en el entorno de otros usuarios conectados al mismo nodo.

Para entender cómo funciona necesitamos conocer las variables de entorno: al ejecutar `module load gcc/11`, la herramienta hace tres cosas principales:

1. **PATH:** añade la ruta del binario al principio del \$PATH.
 - Antes: `/usr/bin:/bin`
 - Después: `*/apps/gcc/11/bin*/usr/bin:/bin`
2. **LD_LIBRARY_PATH:** le dice al cargador dinámico dónde buscar las librerías .so.
 - Nuevo: `/apps/gcc/11/lib64`
3. **MANPATH:** añade la documentación (man gcc).

4.5 Contenedores en HPC: la era de la portabilidad

Los contenedores son la frontera moderna del HPC. Hace 10 años esta unidad no existía y hoy es obligatoria. La mayoría de los trabajos científicos nuevos (especialmente en inteligencia artificial y bioinformática) vienen empaquetados como contenedores.

Existe un desafío para la reproductibilidad. La premisa es: "Esto funciona en mi entorno local". Sin embargo, y a pesar de la existencia de herramientas como los *environment modules*, la replicación precisa de un experimento científico persiste como una tarea de considerable complejidad.

Podemos mencionar dos problemas frecuentes de incompatibilidad:

- Diferencias en el sistema operativo (SO): por ejemplo, un desarrollador utiliza una laptop con Ubuntu 22.04, mientras que el clúster de alto rendimiento (HPC) emplea Rocky Linux 8.
- Conflictos de dependencias y librerías: por ejemplo, una dependencia crítica de Python requiere una versión específica de la librería `glibc` que no está disponible o es una versión anterior en el clúster.

La solución adoptada por la industria (*web/cloud*) para resolver estos problemas fue Docker. Pero el uso de esta herramienta está vetado en la mayoría de los superordenadores por consideraciones de seguridad y arquitectura:

Problemas de seguridad: escalada de privilegios	Consideraciones de arquitectura
Docker funciona mediante un servicio en segundo plano (<i>daemon</i>) que corre como <code>root</code> . Si un usuario tiene permiso para ejecutar comandos de Docker, técnicamente puede montar el sistema de archivos del servidor (<i>/</i>) dentro de su contenedor, modificar archivos del sistema y volverse propietario de la máquina.	Docker está diseñado para microservicios aislados, no para trabajos de cálculo integrados.

Veamos entonces la solución diseñada para nuestro problema:



Apptainer como solución

Apptainer es el nuevo nombre de Singularity bajo la Linux Foundation. Fue creado específicamente para solucionar esto en laboratorios nacionales, como Berkeley Lab.

Veamos sus principales características:

Cierre



¡Felicitaciones, llegaste hasta el final del último módulo del curso!

¿Aprendiste muchas cosas? Esperamos que sí.

Ahora ya casi estás en condiciones de completar la evaluación del curso. Pero antes de eso te sugerimos revisar el [glosario de este módulo](#) y recordá completar los ejercicios.



¡No te vayas sin practicar lo aprendido! Dirigite a la sección de ejercitación haciendo [clic aquí](#).